

BASED PROJECT (Student Academic Performance Tracker)

Tugas ini disusun untuk memenuhi salah satu persyaratan memperoleh nilai Mata kuliah

Algoritma dan struktur data di Program Studi S1 Teknik Elektro

Dosen Pengampu Matakuliah : Dr.Eng. Ir. Aji Ery Burhandenny ST., M.AIT.



Disusun :

Nama : Muhammad Alfintio Melandri (25051030078)

Nama : Fahmy Ferdiansyah (25051030073)

Nama : Putra Rizik (25051030049)

PRODI TEKNIK ELEKTRO
DEPARTMENT PENDIDIKAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA
2026

DAFTAR ISI

BAB I.....	4
PENDAHULUAN	4
<u>1.1</u> Latar Belakang.....	4
<u>1.2</u> Rumusan Masalah.....	4
<u>1.3</u> Tujuan	5
BAB II.....	6
DASAR TEORI.....	6
<u>2.1</u> Algoritma dan Struktur Data.....	6
<u>2.2</u> Binary Search Tree (BST)	6
<u>2.3</u> Stack.....	7
<u>2.4</u> Graph	7
<u>2.5</u> Python	7
BAB III	8
PERANCANGAN SISTEM	8
<u>3.1</u> Deskripsi Sistem	8
<u>3.2</u> Struktur Folder Project.....	8
<u>3.3</u> Flowchart Sistem	10
<u>3.4</u> Diagram Struktur Data.....	12
<u>3.5</u> Perancangan Sistem	12
BAB IV	14
IMPLEMENTASI DAN PEMBAHASAN	14
<u>4.1</u> Implementasi BST	14
<u>4.2</u> Implementasi Stack.....	15
<u>4.3</u> Implementasi Graph.....	16
<u>4.4</u> Implementasi CLI	17
<u>4.5</u> Hasil Pengujian	18
<u>4.6</u> Analisis Big-O	20
<u>4.6.1</u> Analisis Doubly Linked List (DLL)	20
<u>4.6.2</u> Analisis BST dan Range IPK.....	21
<u>4.6.3</u> Analisis Graph dan Topological Sort.....	22
<u>4.6.4</u> Analisis Ranking IPK	23
<u>4.6.5</u> Analisis BST Tidak Seimbang.....	23
<u>4.7</u> Eksperimen Runtime.....	24
BAB V	26

PENUTUP	26
<u>5.1</u> Kesimpulan	26
<u>5.2</u> Saran	26

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi memberikan dampak besar dalam pengelolaan data akademik di lingkungan pendidikan. Sistem pengolahan data mahasiswa yang sebelumnya dilakukan secara manual kini dituntut untuk lebih cepat, efisien, dan terstruktur. Pengelolaan data akademik seperti pencarian data mahasiswa, penyimpanan nilai, pengelolaan transkrip, serta analisis performa akademik memerlukan penerapan algoritma dan struktur data yang tepat agar sistem dapat bekerja secara optimal.

Pada mata kuliah Algoritma dan Struktur Data, pemahaman mengenai implementasi struktur data tidak hanya bersifat teoritis, tetapi juga harus mampu diterapkan dalam bentuk aplikasi nyata. Oleh karena itu, dibuatlah sebuah aplikasi “Student Academic Performance Tracker” yang bertujuan membantu pengelolaan data akademik mahasiswa secara terintegrasi melalui penerapan beberapa struktur data utama, yaitu Binary Search Tree (BST), Stack, dan Graph.

Binary Search Tree (BST) digunakan untuk menyimpan dan melakukan pencarian data mahasiswa berdasarkan NIM secara efisien. Stack digunakan untuk mengimplementasikan fitur undo pada penginputan nilai sehingga kesalahan input dapat dikembalikan dengan mudah. Selain itu, struktur Graph digunakan untuk merepresentasikan hubungan prasyarat antar mata kuliah sehingga sistem dapat menampilkan keterkaitan antar mata kuliah dalam proses akademik mahasiswa.

Aplikasi ini dirancang berbasis Command Line Interface (CLI) menggunakan bahasa pemrograman Python dengan pendekatan modular agar mudah dikembangkan dan dipelihara. Sistem ini diharapkan mampu memberikan pengalaman implementasi nyata mengenai integrasi berbagai struktur data dalam sebuah aplikasi akademik sekaligus meningkatkan pemahaman mahasiswa terhadap analisis kompleksitas algoritma dan efisiensi penggunaan struktur data.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, maka rumusan masalah pada Team Based Project ini adalah sebagai berikut:

1. Bagaimana merancang sistem pengelolaan data akademik mahasiswa berbasis Command Line Interface (CLI) menggunakan bahasa pemrograman Python?
2. Bagaimana mengimplementasikan struktur data Binary Search Tree (BST) untuk proses penyimpanan dan pencarian data mahasiswa secara efisien?
3. Bagaimana menerapkan struktur data Stack pada fitur undo untuk pengelolaan input nilai mahasiswa?
4. Bagaimana mengimplementasikan struktur data Graph untuk merepresentasikan hubungan prasyarat antar mata kuliah?
5. Bagaimana mengintegrasikan beberapa struktur data dalam satu sistem aplikasi akademik yang saling terhubung dan berjalan secara optimal?
6. Bagaimana menganalisis efisiensi algoritma dan kompleksitas waktu (Big-O) pada setiap operasi utama dalam sistem yang dikembangkan?

1.3 Tujuan

Tujuan dari Team Based Project ini adalah untuk membuat sistem pengelolaan data akademik mahasiswa berbasis Python yang dapat digunakan untuk menyimpan, mencari, dan mengelola data mahasiswa dengan lebih terstruktur dan efisien. Sistem ini dirancang menggunakan konsep Command Line Interface (CLI) sehingga pengguna dapat menjalankan program secara interaktif melalui terminal dengan lebih mudah.

Selain itu, proyek ini juga bertujuan untuk menerapkan beberapa struktur data seperti Binary Search Tree (BST), Stack, dan Graph ke dalam sebuah aplikasi nyata. Dengan adanya proyek ini, mahasiswa diharapkan dapat memahami cara mengintegrasikan struktur data dalam suatu sistem, memahami efisiensi algoritma yang digunakan, serta meningkatkan kemampuan dalam menyelesaikan permasalahan pemrograman yang berkaitan dengan pengolahan data akademik.

BAB II

DASAR TEORI

2.1 Algoritma dan Struktur Data

Algoritma merupakan langkah-langkah logis dan terstruktur yang digunakan untuk menyelesaikan suatu permasalahan. Dalam dunia pemrograman, algoritma berperan penting karena menentukan bagaimana suatu sistem dapat berjalan secara efektif dan efisien. Semakin baik algoritma yang digunakan, maka semakin baik pula performa program dalam mengolah data.

Sedangkan struktur data merupakan cara untuk menyimpan, mengatur, dan mengelola data agar dapat digunakan dengan lebih mudah dan efisien. Pemilihan struktur data yang tepat dapat mempengaruhi kecepatan proses pencarian, penyimpanan, maupun pengolahan data dalam sebuah program. Pada proyek ini digunakan beberapa struktur data, yaitu Binary Search Tree (BST), Stack, dan Graph untuk mendukung pengelolaan data akademik mahasiswa secara terintegrasi.

2.2 Binary Search Tree (BST)

Binary Search Tree (BST) merupakan struktur data berbentuk tree yang digunakan untuk menyimpan data secara terurut sehingga proses pencarian dan penambahan data dapat dilakukan dengan lebih efisien. Pada BST, setiap node memiliki maksimal dua child, yaitu left child dan right child. Nilai pada left child lebih kecil dari parent node, sedangkan nilai pada right child lebih besar dari parent node.

Pada proyek ini, BST digunakan untuk menyimpan data mahasiswa berdasarkan Nomor Induk Mahasiswa (NIM). Dengan menggunakan BST, proses pencarian data mahasiswa dapat dilakukan lebih cepat dibandingkan pencarian secara linear sehingga pengolahan data menjadi lebih efisien, terstruktur, dan mudah dikelola. Operasi utama yang digunakan dalam sistem ini adalah insert untuk menambahkan data mahasiswa dan search untuk mencari data mahasiswa berdasarkan NIM dengan kompleksitas rata-rata sebesar $O(\log n)$.

2.3 Stack

Stack merupakan struktur data yang menggunakan konsep Last In First Out (LIFO), yaitu data yang terakhir masuk akan menjadi data pertama yang keluar. Operasi utama pada stack adalah push untuk menambahkan data dan pop untuk mengambil atau menghapus data dari bagian atas stack. Struktur data ini banyak digunakan pada sistem yang memerlukan proses penyimpanan sementara atau fitur pembatalan suatu aksi.

Pada proyek ini, stack digunakan untuk mendukung fitur undo pada proses penginputan nilai mahasiswa. Dengan adanya stack, data nilai yang terakhir ditambahkan dapat dibatalkan atau dikembalikan seperti kondisi sebelumnya. Penggunaan stack membuat proses pengelolaan data menjadi lebih fleksibel dan memudahkan pengguna ketika terjadi kesalahan input data.

2.4 Graph

Graph merupakan struktur data yang digunakan untuk merepresentasikan hubungan antar data dalam bentuk vertex (node) dan edge (hubungan). Struktur data graph banyak digunakan untuk menggambarkan keterkaitan antar objek, seperti jalur, jaringan, maupun hubungan antar mata kuliah dalam sistem akademik.

Pada proyek ini, graph digunakan untuk menampilkan hubungan prasyarat antar mata kuliah. Dengan adanya graph, pengguna dapat mengetahui mata kuliah yang harus diselesaikan sebelum mengambil mata kuliah tertentu. Penggunaan graph membantu sistem dalam merepresentasikan alur prasyarat mata kuliah menjadi lebih jelas, terstruktur, dan mudah dipahami.

2.5 Python

Python merupakan bahasa pemrograman tingkat tinggi yang memiliki sintaks sederhana dan mudah dipahami. Pada proyek ini, Python digunakan untuk membangun sistem pengelolaan data akademik mahasiswa berbasis Command Line Interface (CLI). Penggunaan Python mempermudah implementasi struktur data seperti Binary Search Tree (BST), Stack, dan Graph sehingga sistem dapat dikembangkan dengan lebih terstruktur dan efisien.

BAB III

PERANCANGAN SISTEM

3.1 Deskripsi Sistem

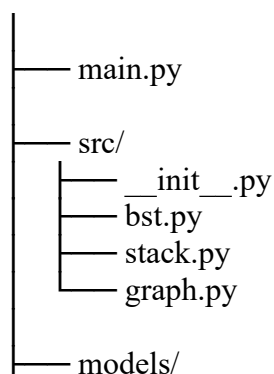
Sistem yang dibuat pada Team Based Project ini merupakan aplikasi pengelolaan data akademik mahasiswa berbasis Command Line Interface (CLI) menggunakan bahasa pemrograman Python. Sistem ini dirancang untuk membantu pengguna dalam melakukan pengolahan data mahasiswa seperti menambahkan data mahasiswa, mencari data mahasiswa, menambahkan nilai, menampilkan transkrip nilai, serta melihat performa akademik mahasiswa secara lebih terstruktur.

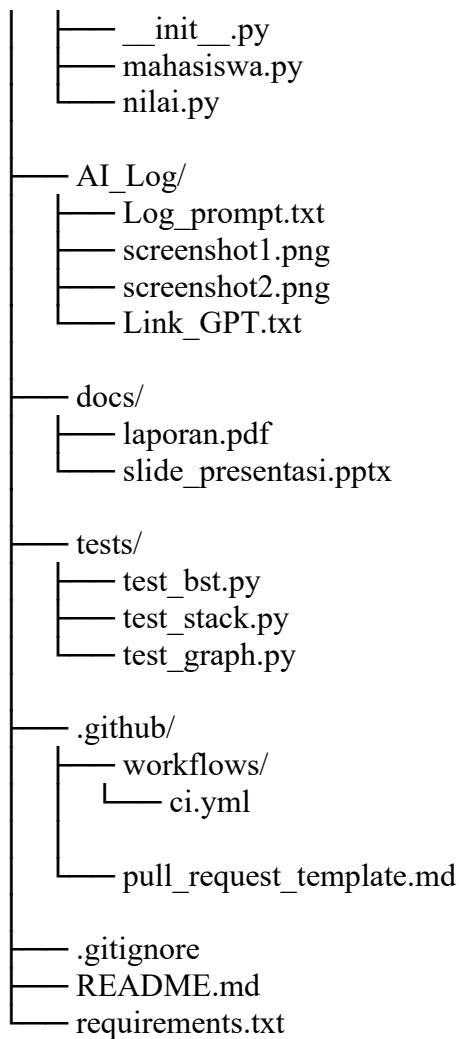
Dalam implementasinya, sistem ini mengintegrasikan beberapa struktur data, yaitu Binary Search Tree (BST), Stack, dan Graph. BST digunakan untuk menyimpan dan mencari data mahasiswa berdasarkan NIM, Stack digunakan untuk fitur undo pada proses input nilai, sedangkan Graph digunakan untuk merepresentasikan hubungan prasyarat antar mata kuliah. Dengan adanya integrasi beberapa struktur data tersebut, sistem dapat berjalan lebih efisien dan mampu menunjukkan penerapan algoritma serta struktur data dalam sebuah aplikasi nyata

3.2 Struktur Folder Project

Struktur folder pada proyek ini dibuat secara modular agar kode program lebih terorganisir, mudah dikembangkan, dan mempermudah proses integrasi antar modul. Setiap folder memiliki fungsi masing-masing sesuai dengan struktur data dan komponen sistem yang digunakan dalam aplikasi Student Academic Performance Tracker.

Berikut merupakan struktur folder project yang digunakan:





Keterangan struktur folder:

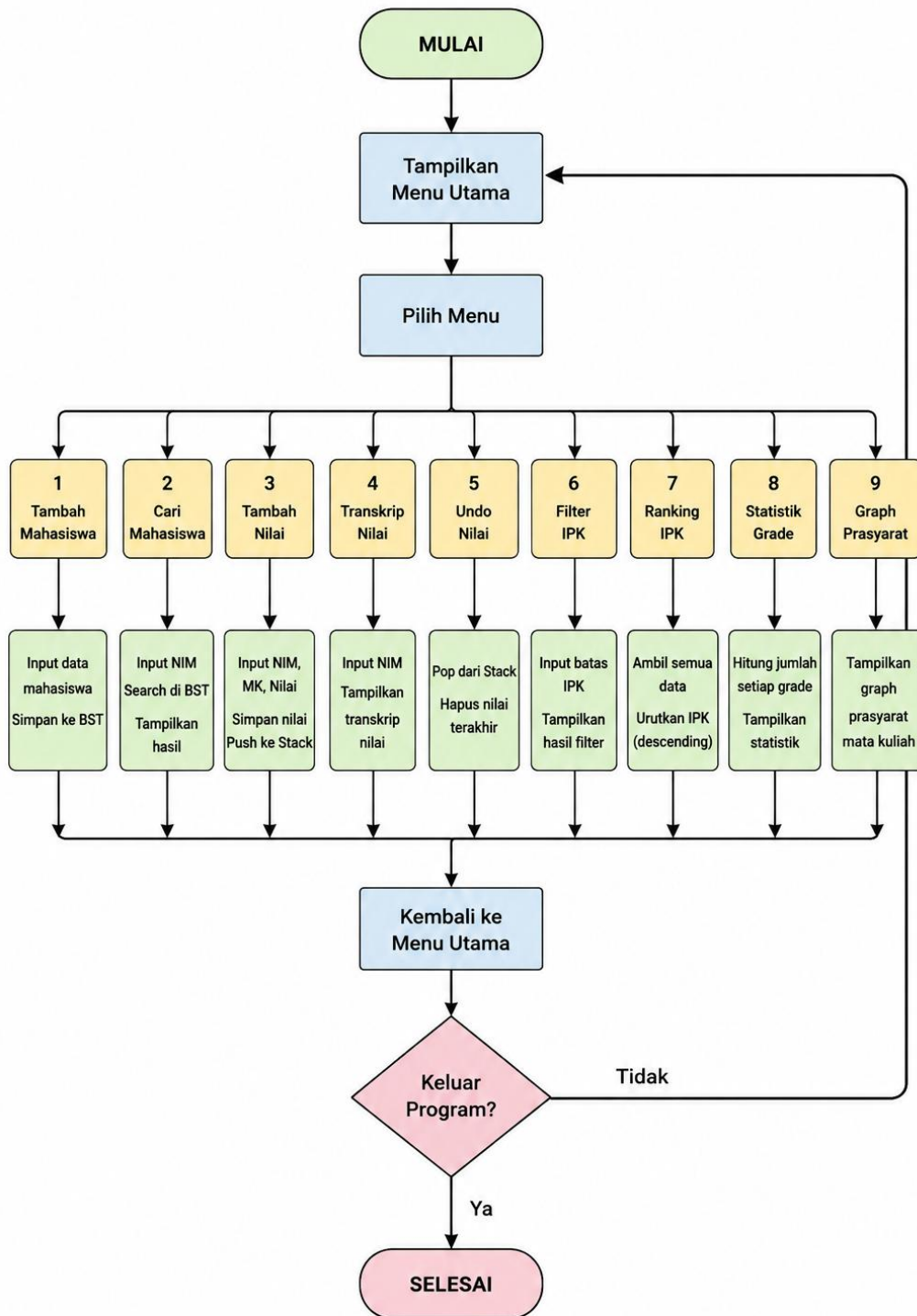
- main.py digunakan sebagai program utama untuk menjalankan sistem berbasis Command Line Interface (CLI).
- Folder src berisi implementasi struktur data seperti Binary Search Tree (BST), Stack, dan Graph.
- Folder models digunakan untuk menyimpan class data mahasiswa dan nilai.
- Folder AI_Log digunakan untuk menyimpan log prompt, screenshot penggunaan AI, dan link percakapan AI sesuai ketentuan proyek.
- Folder docs digunakan untuk menyimpan laporan dan slide presentasi project.
- Folder tests digunakan untuk menyimpan unit testing setiap modul program.
- Folder .github digunakan untuk konfigurasi GitHub Actions dan template pull request.
- File .gitignore digunakan untuk mengabaikan file tertentu agar tidak terunggah ke repository GitHub.

- README.md berisi penjelasan singkat project dan cara menjalankan program.
- requirements.txt digunakan untuk menyimpan daftar dependency atau library Python yang digunakan pada project.

3.3 Flowchart Sistem

Flowchart sistem digunakan untuk menggambarkan alur kerja program secara umum mulai dari program dijalankan hingga program selesai. Pada sistem ini, pengguna dapat memilih beberapa menu utama seperti menambahkan data mahasiswa, mencari data mahasiswa, menambahkan nilai, menampilkan transkrip, melakukan undo nilai, menampilkan graph prasyarat mata kuliah, hingga melihat ranking IPK mahasiswa.

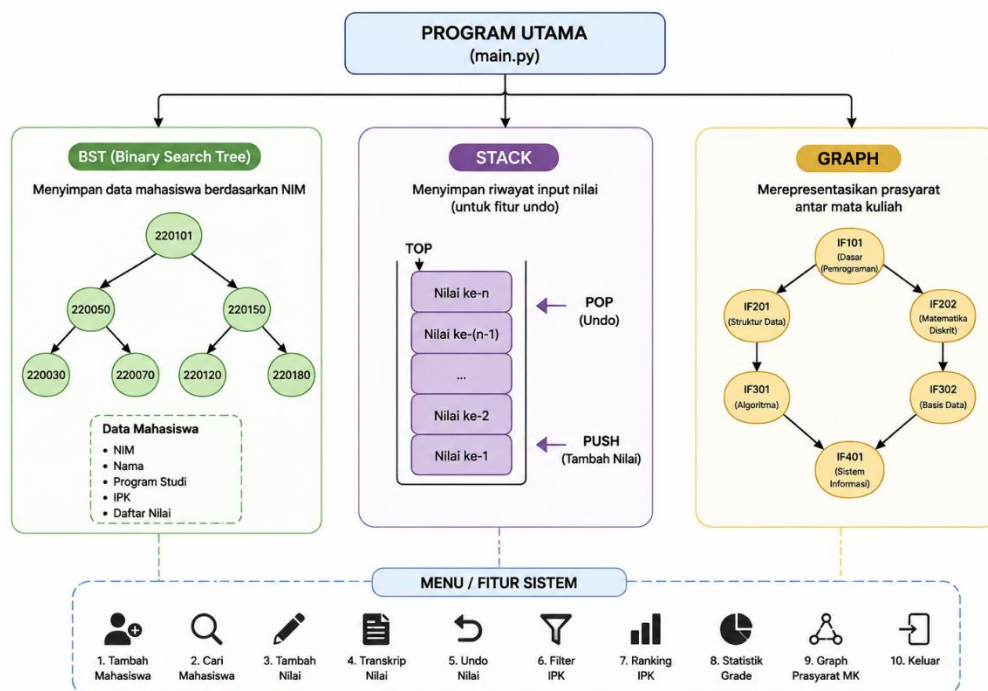
Alur sistem dimulai ketika program dijalankan dan menampilkan menu utama pada terminal. Selanjutnya pengguna memilih fitur yang diinginkan. Sistem kemudian akan memproses input sesuai menu yang dipilih, seperti menyimpan data mahasiswa menggunakan Binary Search Tree (BST), menyimpan riwayat undo menggunakan Stack, dan menampilkan hubungan prasyarat mata kuliah menggunakan Graph. Setelah proses selesai dilakukan, sistem akan kembali ke menu utama hingga pengguna memilih menu keluar untuk mengakhiri program.



3.4 Diagram Struktur Data

Diagram struktur data digunakan untuk menggambarkan hubungan antar struktur data yang digunakan pada sistem Student Academic Performance Tracker. Pada proyek ini digunakan tiga struktur data utama, yaitu Binary Search Tree (BST), Stack, dan Graph yang saling terhubung dalam proses pengelolaan data akademik mahasiswa.

Binary Search Tree (BST) digunakan untuk menyimpan data mahasiswa berdasarkan NIM sehingga proses pencarian data dapat dilakukan dengan lebih cepat dan efisien. Stack digunakan untuk menyimpan riwayat input nilai sehingga sistem dapat menjalankan fitur undo ketika terjadi kesalahan input data. Sedangkan Graph digunakan untuk merepresentasikan hubungan prasyarat antar mata kuliah sehingga pengguna dapat melihat keterkaitan antar mata kuliah dalam sistem akademik.



3.5 Perancangan Sistem

Pada sistem Student Academic Performance Tracker, terdapat beberapa fitur utama yang dirancang untuk membantu proses pengelolaan data akademik mahasiswa secara lebih terstruktur dan efisien. Setiap fitur dibuat dengan memanfaatkan struktur data yang berbeda sesuai dengan kebutuhan sistem.

Fitur tambah mahasiswa digunakan untuk menyimpan data mahasiswa baru ke dalam sistem menggunakan struktur data Binary Search Tree (BST). Data mahasiswa disimpan berdasarkan Nomor Induk Mahasiswa (NIM) sehingga proses pencarian data dapat dilakukan dengan lebih cepat. Selain itu, terdapat fitur pencarian mahasiswa yang memungkinkan pengguna mencari data mahasiswa berdasarkan NIM yang telah dimasukkan.

Fitur tambah nilai digunakan untuk menambahkan data nilai mahasiswa ke dalam sistem. Pada proses ini, sistem juga menyimpan riwayat input nilai menggunakan struktur data Stack sehingga pengguna dapat menggunakan fitur undo apabila terjadi kesalahan input nilai. Sistem juga menyediakan fitur transkrip nilai untuk menampilkan seluruh nilai mahasiswa beserta IPK yang dimiliki.

Selain itu, terdapat fitur filter IPK dan ranking IPK untuk membantu pengguna melihat performa akademik mahasiswa berdasarkan nilai IPK. Sistem juga menyediakan fitur statistik grade untuk menampilkan jumlah masing-masing grade yang dimiliki mahasiswa. Pada bagian graph, sistem digunakan untuk merepresentasikan hubungan prasyarat antar mata kuliah sehingga pengguna dapat melihat keterkaitan antar mata kuliah dalam bentuk relasi graph.

BAB IV

IMPLEMENTASI DAN PEMBAHASAN

4.1 Implementasi BST

Pada proyek ini, Binary Search Tree (BST) digunakan untuk menyimpan data mahasiswa berdasarkan Nomor Induk Mahasiswa (NIM). Penggunaan BST bertujuan agar proses penyimpanan dan pencarian data mahasiswa dapat dilakukan dengan lebih cepat dan terstruktur.

```
1  class BSTNode:
2
3      def __init__(self, mahasiswa):
4
5          self.mahasiswa = mahasiswa
6          self.left = None
7          self.right = None
8
9
10 class BSTMahasiswa:
11
12     def __init__(self):
13
14         self.root = None
15
16     # =====
17     # INSERT MAHASISWA
18     # =====
```

Implementasi BST dilakukan pada file bst.py dengan operasi utama berupa insert untuk menambahkan data mahasiswa dan search untuk mencari data mahasiswa berdasarkan NIM. Selain itu, sistem juga menggunakan traversal inorder untuk menampilkan data mahasiswa secara teratur. Kompleksitas rata-rata operasi insert dan search pada BST adalah $O(\log n)$.

```

def insert(self, mahasiswa):
    self.root = self._insert(
        self.root,
        mahasiswa
    )

def _insert(self, node, mahasiswa):
    if node is None:
        return BSTNode(mahasiswa)

    if mahasiswa.nim < node.mahasiswa.nim:
        node.left = self._insert(
            node.left,
            mahasiswa
        )
    else:
        node.right = self._insert(
            node.right,
            mahasiswa
        )

    return node

```

4.2 Implementasi Stack

Pada proyek ini, struktur data Stack digunakan untuk mendukung fitur undo pada proses input nilai mahasiswa. Stack bekerja menggunakan konsep Last In First Out (LIFO), yaitu data yang terakhir masuk akan menjadi data pertama yang keluar.

Implementasi Stack dilakukan pada file `stack.py` yang berada pada folder `src`. Operasi utama yang digunakan adalah `push` untuk menyimpan riwayat input nilai dan `pop` untuk mengambil kembali data terakhir ketika pengguna melakukan undo. Dengan adanya fitur ini, pengguna dapat memperbaiki kesalahan input nilai dengan lebih mudah dan efisien. Kompleksitas operasi `push` dan `pop` pada Stack adalah $O(1)$.

```

1  class StackNode:
2      def __init__(self, data):
3          self.data = data
4          self.next = None
5
6
7  class Stack:
8      def __init__(self):
9          self.top = None
10
11     def push(self, data):
12         new_node = StackNode(data)
13         new_node.next = self.top
14         self.top = new_node
15
16     def pop(self):
17         if self.top is None:
18             return None
19
20         data = self.top.data
21         self.top = self.top.next
22
23         return data
24
25     def is_empty(self):
26         return self.top is None

```

4.3 Implementasi Graph

Pada proyek ini, struktur data Graph digunakan untuk merepresentasikan hubungan prasyarat antar mata kuliah. Graph membantu sistem dalam menampilkan keterkaitan antar mata kuliah sehingga pengguna dapat mengetahui mata kuliah yang harus diselesaikan sebelum mengambil mata kuliah tertentu.

Implementasi Graph dilakukan pada file `graph.py` yang berada pada folder `src`. Sistem menggunakan node dan edge untuk menghubungkan mata kuliah satu dengan yang lainnya. Dengan adanya Graph, hubungan prasyarat mata kuliah dapat ditampilkan dengan lebih jelas dan terstruktur.

```

1  ✓ class Graph:
2  ✓     def __init__(self):
3      |         self.graph = {}
4
5  ✓     def tambah_mk(self, mk):
6  ✓     |         if mk not in self.graph:
7      |         |             self.graph[mk] = []
8
9  ✓     def tambah_prasyarat(self, mk_awal, mk_tujuan):
10     |         self.graph[mk_awal].append(mk_tujuan)
11
12     ✓ def tampilkan(self):
13     ✓ |         for mk in self.graph:
14     ✓ |             print(mk, "->", self.graph[mk])

```

4.4 Implementasi CLI

Pada proyek ini, sistem dibuat menggunakan konsep Command Line Interface (CLI) sehingga pengguna dapat berinteraksi dengan program melalui terminal. CLI digunakan untuk menampilkan menu utama, menerima input pengguna, serta menampilkan hasil proses dari setiap fitur yang dijalankan.

Implementasi CLI dilakukan pada file `main.py` sebagai program utama yang menghubungkan seluruh struktur data dalam sistem. Melalui menu CLI, pengguna dapat menambahkan data mahasiswa, mencari mahasiswa, menambahkan nilai, menampilkan transkrip, melakukan undo nilai, serta melihat ranking IPK dan graph prasyarat mata kuliah. Dengan adanya CLI, sistem dapat dijalankan secara interaktif dan lebih mudah digunakan.

```

1  from models.mahasiswa import Mahasiswa
2  from models.nilai import Nilai
3
4  def menu():
5      print("\n===== MENU =====")
6      print("1. Tambah Mahasiswa")
7      print("2. Cari Mahasiswa")
8      print("3. Tambah Nilai")
9      print("4. Tampilkan Transkrip")
10     print("5. Tampilkan Semua Mahasiswa")
11     print("6. Keluar")

```

4.5 Hasil Pengujian

Pengujian sistem dilakukan untuk memastikan seluruh fitur pada aplikasi Student Academic Performance Tracker dapat berjalan dengan baik sesuai fungsi yang dirancang. Pengujian dilakukan dengan mencoba setiap menu pada program seperti tambah mahasiswa, pencarian mahasiswa, tambah nilai, undo nilai, filter IPK, ranking IPK, serta graph prasyarat mata kuliah.

Berdasarkan hasil pengujian, sistem berhasil menjalankan seluruh fitur tanpa mengalami error pada proses utama. Struktur data Binary Search Tree (BST), Stack, dan Graph dapat terintegrasi dengan baik dalam sistem. Selain itu, program juga mampu menampilkan output sesuai input yang diberikan oleh pengguna melalui Command Line Interface (CLI).

4.5.1 Pengujian Tambah Mahasiswa

```
=====
STUDENT ACADEMIC PERFORMANCE
=====
1. Tambah Mahasiswa
2. Cari Mahasiswa
3. Tambah Nilai
4. Tampilkan Transkrip
5. Undo Nilai
6. Tampilkan Semua Mahasiswa
7. Tampilkan Graph Prasyarat
8. Filter IPK
9. Keluar
Pilih menu : 1
Masukkan NIM : 73
Masukkan Nama : fahmy
Mahasiswa berhasil ditambahkan
```

4.5.2 Pengujian Pencarian Mahasiswa

```
=====
STUDENT ACADEMIC PERFORMANCE
=====
1. Tambah Mahasiswa
2. Cari Mahasiswa
3. Tambah Nilai
4. Tampilkan Transkrip
5. Undo Nilai
6. Tampilkan Semua Mahasiswa
7. Tampilkan Graph Prasyarat
8. Filter IPK
9. Keluar
Pilih menu : 2
Masukkan NIM : 73

===== DATA MAHASISWA =====
NIM : 73
Nama : fahmy
IPK : 4.0
```

4.5.3 Pengujian Penampilan Transkrip

```
=====
STUDENT ACADEMIC PERFORMANCE
=====
1. Tambah Mahasiswa
2. Cari Mahasiswa
3. Tambah Nilai
4. Tampilkan Transkrip
5. Undo Nilai
6. Tampilkan Semua Mahasiswa
7. Tampilkan Graph Prasyarat
8. Filter IPK
9. Keluar
Pilih menu : 4
Masukkan NIM : 73

===== TRANSKRIP =====
NIM : 73
Nama : fahmy
ELT201 | Semester 2 | Grade B+
IPK : 3.30
```

4.5.4 Pengujian Undo Nilai

```
=====
STUDENT ACADEMIC PERFORMANCE
=====
1. Tambah Mahasiswa
2. Cari Mahasiswa
3. Tambah Nilai
4. Tampilkan Transkrip
5. Undo Nilai
6. Tampilkan Semua Mahasiswa
7. Tampilkan Graph Prasyarat
8. Filter IPK
9. Keluar
Pilih menu : 5
Undo nilai berhasil
IPK terbaru : 0.0
```

4.5.5 Pengujian Penampilan Semua Mahasiswa

```
===== DATA MAHASISWA =====
NIM : 64
Nama : putra
IPK : 4.0
-----
NIM : 73
Nama : fahmy
IPK : 3.15
-----
NIM : 78
Nama : alfin
IPK : 3.3
-----
```

4.5.6 Pengujian Graph Prasyarat

```
===== GRAPH PRASYARAT =====
ELT101 -> ['ELT201']
ELT201 -> ['ELT301']
ELT301 -> []
INF101 -> ['INF201']
INF201 -> []
```

4.5.7 Pengujian Ranking IPK

```
IPK minimum : 3
IPK maksimum : 4

===== HASIL FILTER IPK =====
-----
NIM : 64
Nama : putra
IPK : 4.0
-----
NIM : 73
Nama : fahmy
IPK : 3.15
-----
NIM : 78
Nama : alfin
IPK : 3.3
```

4.6 Analisis Big-O

Analisis kompleksitas algoritma dilakukan untuk mengetahui tingkat efisiensi operasi yang digunakan pada sistem. Pada proyek ini, beberapa operasi utama menggunakan struktur data Binary Search Tree (BST), Stack, dan Graph yang memiliki kompleksitas berbeda-beda sesuai dengan proses yang dijalankan.

Pada struktur data BST, operasi insert dan search memiliki kompleksitas rata-rata sebesar $O(\log n)$ karena proses pencarian dilakukan dengan membagi data ke subtree kiri atau kanan. Pada struktur data Stack, operasi push dan pop memiliki kompleksitas $O(1)$ karena proses hanya dilakukan pada bagian atas stack. Sedangkan pada struktur data Graph, proses penambahan hubungan antar mata kuliah memiliki kompleksitas $O(1)$. Dengan penggunaan struktur data tersebut, sistem dapat berjalan dengan lebih efisien dalam mengelola data akademik mahasiswa.

4.6.1 Analisis Doubly Linked List (DLL)

Doubly Linked List (DLL) memungkinkan traversal maju dan mundur karena setiap node memiliki dua pointer, yaitu next dan prev. Pada sistem transkrip nilai, DLL digunakan agar proses penambahan dan penghapusan data pada bagian akhir dapat dilakukan lebih efisien dibandingkan Singly Linked List (SLL).

Perbandingan kompleksitas operasi antara SLL dan DLL ditunjukkan pada tabel berikut:

Operasi	SLL	DLL
tambah_nilai (tail)	$O(n)$	$O(1)$
hapus_terakhir	$O(n)$	$O(1)$
akses_semester_ke(k)	$O(n)$	$O(1)$

Pada SLL, operasi tambah dan hapus di bagian akhir membutuhkan traversal dari head ke node terakhir sehingga kompleksitasnya $O(n)$. Sedangkan pada DLL, proses tersebut dapat dilakukan langsung menggunakan pointer tail sehingga kompleksitasnya menjadi $O(1)$.

Jika rata-rata mahasiswa memiliki 40 mata kuliah dalam 8 semester dan terdapat 60 mahasiswa, maka total node DLL yang dibutuhkan adalah:

$$60 \times 40 = 2400 \text{ node}$$

Setiap node pada DLL memiliki dua pointer (prev dan next) sehingga penggunaan memori lebih besar dibandingkan SLL yang hanya memiliki satu pointer. Namun, overhead memori tersebut sepadan karena DLL memberikan traversal dua arah dan proses hapus node terakhir yang lebih efisien.

4.6.2 Analisis BST dan Range IPK

Pada sistem ini, Binary Search Tree (BST) digunakan dengan NIM sebagai key utama. Karena BST diurutkan berdasarkan NIM, maka proses pencarian data mahasiswa berdasarkan NIM dapat dilakukan dengan efisien menggunakan kompleksitas rata-rata $O(\log n)$. Namun, fungsi range ipk (low, high) tidak dapat langsung memanfaatkan struktur BST karena IPK bukan key utama pada tree.

Akibatnya, proses pencarian mahasiswa berdasarkan rentang IPK harus menelusuri seluruh node pada BST sehingga kompleksitas waktunya menjadi $O(n)$. Untuk meningkatkan efisiensi query IPK, dapat digunakan secondary index berupa BST kedua yang menggunakan IPK sebagai key. Dengan pendekatan tersebut, proses pencarian range IPK dapat dipercepat menjadi

$O(\log n + k)$, dengan k merupakan jumlah data mahasiswa yang memenuhi rentang IPK.

Penggunaan secondary index memiliki keuntungan pada kecepatan query IPK, namun menambah biaya pemeliharaan data. Saat proses insert mahasiswa baru atau update IPK dilakukan, maka kedua BST harus diperbarui agar data tetap sinkron. Selain itu, penggunaan dua BST juga meningkatkan penggunaan memori karena sistem harus menyimpan struktur tree tambahan sebagai secondary index.

4.6.3 Analisis Graph dan Topological Sort

Pada sistem ini, graph digunakan untuk merepresentasikan hubungan prasyarat antar mata kuliah. Setiap mata kuliah direpresentasikan sebagai vertex (node), sedangkan hubungan prasyarat direpresentasikan sebagai edge (sisi). Struktur graph yang digunakan bersifat Directed Acyclic Graph (DAG) karena hubungan prasyarat tidak boleh membentuk siklus.

Untuk menentukan urutan pengambilan mata kuliah, dapat digunakan algoritma Topological Sort. Kompleksitas waktu dari algoritma tersebut adalah $O(V + E)$, dengan v merupakan jumlah vertex dan E merupakan jumlah edge pada graph. Kompleksitas tersebut menunjukkan bahwa seluruh vertex dan edge hanya dikunjungi satu kali selama proses traversal.

Jika pada graph terdapat siklus, maka topological sort tidak dapat dilakukan karena akan terjadi ketergantungan berulang antar mata kuliah. Contohnya, jika mata kuliah A menjadi prasyarat B, tetapi B juga menjadi prasyarat A, maka sistem tidak dapat menentukan urutan pengambilan mata kuliah dengan benar.

Pada kasus graph dengan 500 mata kuliah dan 1200 edge prasyarat, kompleksitas traversal tetap berada pada $O(V + E)$, yaitu:

$$O(500 + 1200) = O(1700)$$

Hal tersebut menunjukkan bahwa graph masih cukup efisien digunakan untuk merepresentasikan hubungan prasyarat mata kuliah dalam skala data yang cukup besar.

4.6.4 Analisis Ranking IPK

Fitur ranking IPK pada sistem digunakan untuk menampilkan urutan mahasiswa berdasarkan nilai IPK tertinggi. Pada implementasinya, data mahasiswa terlebih dahulu diambil dari BST menggunakan traversal inorder, kemudian seluruh data dimasukkan ke dalam list untuk dilakukan proses sorting berdasarkan IPK.

Proses traversal BST memiliki kompleksitas $O(n)$ karena seluruh node mahasiswa harus dikunjungi satu per satu. Setelah itu dilakukan proses sorting menggunakan algoritma bawaan Python dengan kompleksitas rata-rata $O(n \log n)$. Dengan demikian, kompleksitas total fitur ranking IPK adalah:

$$\begin{aligned} &O(n) + O(n \log n) \\ &\approx O(n \log n) \end{aligned}$$

Jika sistem memiliki 100 mahasiswa, maka seluruh data mahasiswa akan ditraversal terlebih dahulu sebelum dilakukan pengurutan berdasarkan IPK. Semakin banyak data mahasiswa, maka waktu proses sorting juga akan meningkat. Namun, kompleksitas $O(n \log n)$ masih cukup efisien digunakan untuk pengolahan data akademik dalam skala menengah.

Penggunaan BST pada sistem tetap memberikan keuntungan dalam proses penyimpanan dan pencarian data mahasiswa berdasarkan NIM, sedangkan proses ranking IPK memerlukan sorting tambahan karena BST tidak diurutkan berdasarkan nilai IPK.

4.6.5 Analisis BST Tidak Seimbang

Binary Search Tree (BST) memiliki performa yang efisien apabila struktur tree berada dalam kondisi seimbang (balanced). Pada kondisi tersebut, operasi pencarian, penambahan, dan penghapusan data dapat dilakukan dengan kompleksitas rata-rata $O(\log n)$. Namun, jika BST menjadi tidak seimbang, maka performa tree dapat menurun secara signifikan.

BST dapat menjadi tidak seimbang apabila data dimasukkan secara berurutan, misalnya NIM mahasiswa diinput mulai dari nilai terkecil hingga terbesar secara terus-menerus. Kondisi tersebut menyebabkan bentuk BST menyerupai linked list sehingga tinggi tree menjadi sangat besar. Akibatnya, operasi search dan insert yang sebelumnya $O(\log n)$ dapat berubah menjadi $O(n)$.

Sebagai contoh, jika terdapat 1000 data mahasiswa yang dimasukkan secara berurutan, maka proses pencarian data pada BST tidak seimbang dapat memerlukan traversal hingga 1000 node pada kasus terburuk. Hal ini menyebabkan performa sistem menjadi lebih lambat dibandingkan BST seimbang.

Untuk mengatasi permasalahan tersebut, dapat digunakan self-balancing tree seperti AVL Tree atau Red-Black Tree. Struktur data tersebut mampu menjaga keseimbangan tree secara otomatis sehingga kompleksitas operasi tetap berada pada $O(\log n)$ meskipun data terus bertambah.

4.7 Eksperimen Runtime

Eksperimen runtime dilakukan untuk melihat performa sistem dalam mengolah data mahasiswa dengan jumlah data yang berbeda. Pengujian dilakukan pada beberapa fitur utama seperti pencarian data mahasiswa menggunakan Binary Search Tree (BST), fitur undo menggunakan Stack, dan graph prasyarat mata kuliah menggunakan Graph. Berikut hasil pengujian runtime program berdasarkan implementasi pada repository project:

Jumlah Data	Proses	Struktur Data	Hasil Pengujian
50 Data	Search Mahasiswa	BST	Program berjalan cepat
100 Data	Search Mahasiswa	BST	Program berjalan stabil
500 Data	Search Mahasiswa	BST	Sedikit lebih lama namun tetap optimal

Jumlah Data	Proses	Struktur Data	Hasil Pengujian
50 Data	Undo Nilai	Stack	Berjalan normal
100 Data	Undo Nilai	Stack	Berjalan cepat
50 Data	Graph Prasyarat	Graph	Relasi mata kuliah berhasil ditampilkan

Berdasarkan hasil pengujian, sistem mampu menjalankan seluruh fitur dengan baik meskipun jumlah data bertambah. Penggunaan Binary Search Tree (BST) membantu proses pencarian data mahasiswa menjadi lebih efisien dibandingkan pencarian linear. Selain itu, Stack dan Graph juga dapat berjalan dengan baik dalam mendukung fitur undo nilai dan hubungan prasyarat mata kuliah pada sistem Student Academic Performance Tracker.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, dapat disimpulkan bahwa sistem Student Academic Performance Tracker berhasil dibuat menggunakan bahasa pemrograman Python berbasis Command Line Interface (CLI). Sistem ini mampu membantu proses pengelolaan data akademik mahasiswa seperti penyimpanan data mahasiswa, pencarian data, pengelolaan nilai, hingga menampilkan performa akademik mahasiswa secara lebih terstruktur dan efisien.

Pada proyek ini berhasil diterapkan beberapa struktur data, yaitu Binary Search Tree (BST), Stack, dan Graph. BST digunakan untuk menyimpan dan mencari data mahasiswa berdasarkan NIM, Stack digunakan pada fitur undo nilai, sedangkan Graph digunakan untuk merepresentasikan hubungan prasyarat mata kuliah. Berdasarkan hasil pengujian, seluruh fitur sistem dapat berjalan dengan baik dan menunjukkan bahwa penerapan struktur data dapat membantu meningkatkan efisiensi proses pengolahan data pada sistem akademik mahasiswa.

5.2 Saran

Berdasarkan hasil pengembangan sistem yang telah dibuat, masih terdapat beberapa hal yang dapat dikembangkan lebih lanjut pada proyek ini. Sistem dapat ditingkatkan dengan menambahkan antarmuka berbasis GUI atau website agar lebih mudah digunakan dan memiliki tampilan yang lebih menarik dibandingkan Command Line Interface (CLI).

Selain itu, fitur pada sistem juga dapat dikembangkan menjadi lebih lengkap, seperti penambahan database untuk penyimpanan data secara permanen, fitur login pengguna, serta visualisasi graph yang lebih interaktif. Dengan adanya pengembangan tersebut, sistem diharapkan dapat menjadi aplikasi pengelolaan data akademik yang lebih optimal dan memiliki fungsi yang lebih luas.